

What Matters for Coordinated Multi-Agent RL on SMAC: An Ablation Study of Algorithm, Memory, and Reward

Cedric Orton-Urbina (ceo29)
CS 5756, Cornell University, Spring 2026

Abstract

We train cooperative multi-agent reinforcement learning agents on the StarCraft Multi-Agent Challenge (SMAC) 3m micromanagement scenario and conduct a controlled three-axis ablation study to determine which design choices most strongly affect performance. The three axes are the learning algorithm (QMIX vs. MAPPO), the agent’s access to temporal history (GRU vs. MLP), and the reward signal (default dense vs. sparse). At a fixed budget of 525,000 environment steps per run and three seeds per condition, QMIX with a GRU agent and dense reward reaches a 96.0% test win rate, the corresponding MLP variant reaches 96.9%, MAPPO under the same conditions reaches a mean of 44.2% with a 32.7 percentage point standard deviation across seeds, and the sparse-reward variant remains at 0.0% for every seed. These results show that reward density has the largest effect on learning in this setting, that off-policy QMIX is substantially more stable than on-policy MAPPO under a limited sample budget, and that recurrence provides little benefit on 3m.

Keywords: Multi-Agent Reinforcement Learning, SMAC, QMIX, MAPPO, Ablation Study

1 Introduction

Cooperative multi-agent reinforcement learning (MARL) is shaped by three central difficulties. The first is **partial observability**. Each agent receives only a local view of the environment, so decision-making must proceed from incomplete information. The second is **non-stationarity**. Because all agents learn simultaneously, the effective environment faced by any one agent changes throughout training. The third is **credit assignment**. The team receives a shared reward, and the learner must infer which agents and actions contributed to that outcome. These challenges interact closely, and even small benchmark tasks can require reliable coordination to solve.

The StarCraft Multi-Agent Challenge (SMAC) [3] is a standard benchmark for this setting. It removes macro-level strategy and economy from StarCraft II and reduces the problem to a single combat encounter against a fixed scripted opponent. Each allied unit is controlled by a separate learning agent. The 3m scenario, which consists of three Marines fighting three enemy Marines, is small enough to support repeated controlled experiments while still requiring coordination in target selection, focus fire, and movement.

This paper studies which design choices matter most in that setting. We run a controlled ablation in which the scenario is held fixed and exactly one factor is changed in each comparison:

- **Algorithm** (A1): off-policy value decomposition with QMIX [2] versus on-policy actor-critic with MAPPO [5].

- **History (A2):** a recurrent per-agent policy using a GRU versus a feed-forward MLP that does not explicitly represent history.
- **Reward (A3):** SMAC’s default dense reward versus a purely sparse terminal reward.

The results show that the reward signal is the dominant factor in this experiment. Replacing dense reward with sparse reward reduces final test win rate from about 96% to 0% across three seeds, with no recovery during the full 525,000-step training budget. The sparse-reward agents still learn a consistent behavior, but that behavior delays defeat instead of producing wins. The comparison between GRU and MLP shows no meaningful difference on 3m, which suggests that the instantaneous observation already contains most of the information needed for near-optimal control. The comparison between QMIX and MAPPO shows that MAPPO can perform competitively on its best seed, but it exhibits much larger variance across seeds under the same sample budget.

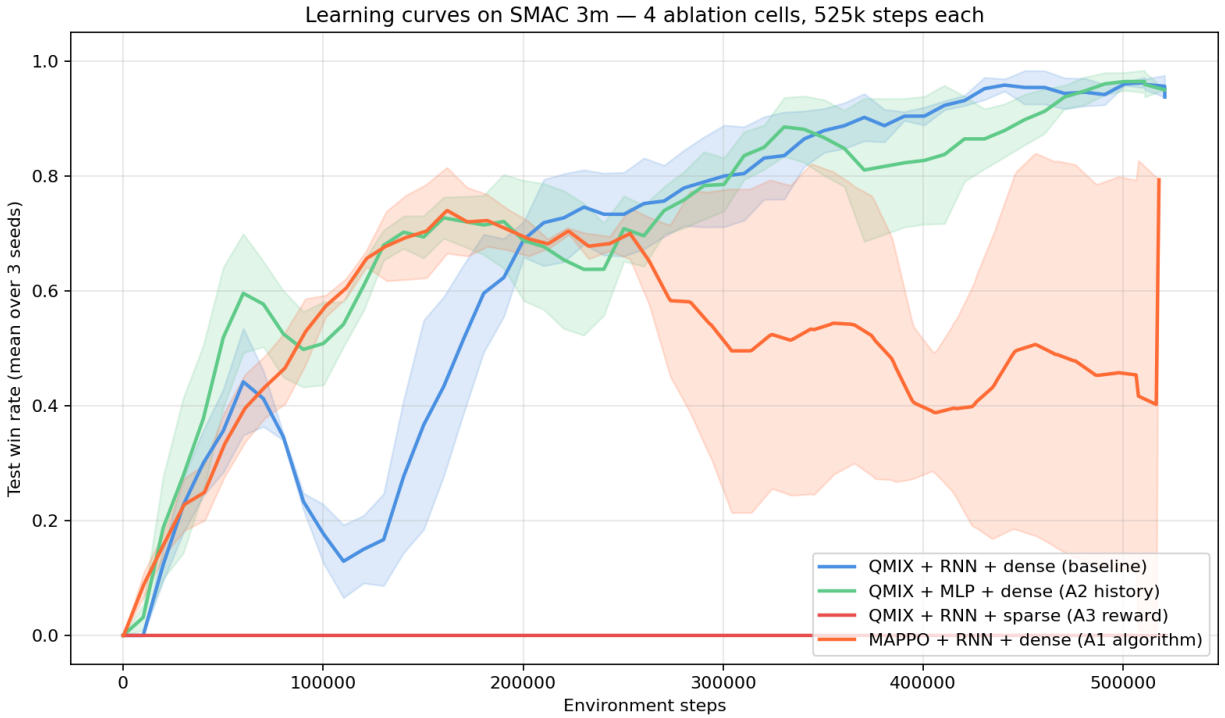


Figure 1: Test win rate as a function of environment steps for all four ablation settings. Lines show the mean across three seeds, and shaded regions show one standard deviation. QMIX with a GRU agent and dense reward (blue) and the corresponding MLP variant (green) converge to near-ceiling performance within about 400k steps. MAPPO (orange) reaches roughly 70% around 200k steps and then enters a high-variance collapse-and-recovery phase between 250k and 500k steps. Sparse-reward QMIX (red) remains at zero throughout training.

2 Related Work

The benchmark. SMAC was introduced by Samvelyan et al. [3] as a cooperative MARL benchmark built on the StarCraft II Learning Environment. It provides a family of micromanagement scenarios in which each allied unit is controlled by a separate agent under centralized training with

decentralized execution. The 3m scenario used here is the smallest map in the benchmark, but it still requires coordinated target selection and action timing.

Value decomposition methods. Value Decomposition Networks (VDN) [4] proposed factorizing a joint action-value function into a sum of per-agent action-value functions, which makes centralized training compatible with decentralized execution. QMIX [2] extended this idea by introducing a learned monotonic mixing network conditioned on the global state. QMIX remains one of the standard baselines on SMAC, and we use it here as the reference method.

On-policy multi-agent actor-critic. MAPPO [5] adapts proximal policy optimization to the multi-agent setting by sharing policy parameters across agents and using a centralized critic. The original work showed that MAPPO can be competitive with off-policy methods on SMAC when training conditions are chosen carefully. Our results are consistent with that result on favorable seeds, although we find substantially higher seed variance under a restricted sample budget.

The codebase. EPyMARL [1] is a maintained reimplement of PyMARL that provides QMIX, VDN, IQL, MAPPO, COMA, and related algorithms within a unified configuration framework. We use EPyMARL and adopt its default hyperparameters for the algorithms evaluated in this study.

3 Problem Formulation

We adopt the cooperative Decentralized Partially Observable Markov Decision Process (Dec-POMDP) formulation standard for SMAC. At each timestep t , the environment is in a global state $s_t \in \mathcal{S}$, and each of N agents $i \in \{1, \dots, N\}$ receives a local observation $o_t^i \in \mathcal{O}$ that depends on s_t but does not in general reveal it completely. Each agent selects an action $a_t^i \in \mathcal{A}$ according to a policy $\pi^i(a^i | h_t^i)$, where $h_t^i = (o_0^i, a_0^i, \dots, o_t^i)$ is the agent’s action-observation history. The joint action $\mathbf{a}_t = (a_t^1, \dots, a_t^N)$ produces the next state and a shared reward $r_t \in \mathbb{R}$. The team objective is to maximize the expected discounted return

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t r_t \right].$$

For 3m, $N = 3$ allied Marines. The per-agent action space is discrete and small: no-op, stop, movement in the four cardinal directions, and attack actions for visible enemies. The environment is partially observable because each agent receives features only for units within its sight radius. The team is trained against a fixed scripted opponent provided by StarCraft II at difficulty 7. Both QMIX and MAPPO are trained under centralized training with decentralized execution. A global state s_t is available during training, through either QMIX’s mixing network or MAPPO’s centralized critic, while each agent acts at test time from h_t^i alone.

4 Method

4.1 QMIX

QMIX learns a per-agent action-value function $Q_i(\tau^i, a^i)$ implemented with a parameter-shared neural network. At each timestep, agent i evaluates Q_i over its action set from its history τ^i , encoded by a GRU, and selects an action using ϵ -greedy exploration. A mixing network then combines the per-agent action-values $\{Q_i\}$ with the global state s_t under a monotonicity constraint to produce a joint action-value estimate. This structure allows temporal-difference targets defined

at the team level to update the agent-level networks while preserving decentralized greedy action selection at execution time. We use the EPyMARL defaults, including a replay buffer of 5000 episodes, ϵ annealed from 1.0 to 0.05 over 50,000 steps, target-network updates every 200 episodes, and Adam with learning rate 5×10^{-4} .

4.2 MAPPO

MAPPO replaces the off-policy value-based learner with an on-policy actor-critic method. The policy is shared across agents and outputs an action distribution for each agent. A centralized critic conditioned on the global state estimates $V(s_t)$, which is used with generalized advantage estimation. The PPO clipped-ratio objective is applied at the agent level. Rollouts are collected from ten parallel environments per update, and each rollout is reused for four optimization epochs. All remaining hyperparameters follow the EPyMARL defaults.

4.3 History ablation

To study the effect of explicit temporal memory, we toggle the recurrence flag in the per-agent network from True to False. The recurrent setting uses a GRU, while the non-recurrent setting uses a single linear layer followed by ReLU. The mixing network and the rest of the training pipeline remain unchanged. In principle, a Dec-POMDP requires access to action-observation history for optimal decision-making. This ablation tests whether that requirement has a measurable practical effect on 3m.

4.4 Reward ablation

SMAC’s default reward is dense. At each step, the team receives a small positive signal for damage dealt to enemies, a smaller penalty for damage taken, and large terminal bonuses for kills and victory. In the sparse variant, all intermediate rewards are removed and the team receives only +1 for a win and −1 for a loss at episode termination.

4.5 Compute and sample budget

Each of the four experimental settings is trained with three independent random seeds. Every seed is run for 525,000 environment steps, with test evaluation every 10,000 steps using 32 evaluation episodes and $\epsilon = 0$. Training was performed on a single NVIDIA L4 GPU on Google Cloud Platform. The QMIX conditions each required about 70 minutes per seed, while MAPPO, which benefits from ten parallel environments, required about 25 minutes per seed.

5 Experiments

5.1 Experimental setup

All experiments use the SMAC 3m map: three allied Marines versus three enemy Marines on a small flat arena, with the enemy team controlled by the built-in StarCraft II scripted AI at difficulty 7. The training pipeline is the EPyMARL implementation [1] (v2.0.0) with SC2 build 4.6.2, which is the version pinned by SMAC. All four settings share the same defaults except for the single factor under ablation.

5.2 Quantitative results

Final test win rate, measured as the mean over the last 32 evaluation episodes after 525,000 environment steps, is summarized in Table 2 and visualized in Figure 2. Learning curves for all four

Table 1: The four ablation settings. Each row changes one factor relative to the baseline while holding the other two fixed.

Cell	Algorithm	Agent	Reward	Seeds
Baseline	QMIX	GRU	Dense	1, 2, 3
A1	MAPPO	GRU	Dense	1, 2, 3
A2	QMIX	MLP	Dense	1, 2, 3
A3	QMIX	GRU	Sparse	1, 2, 3

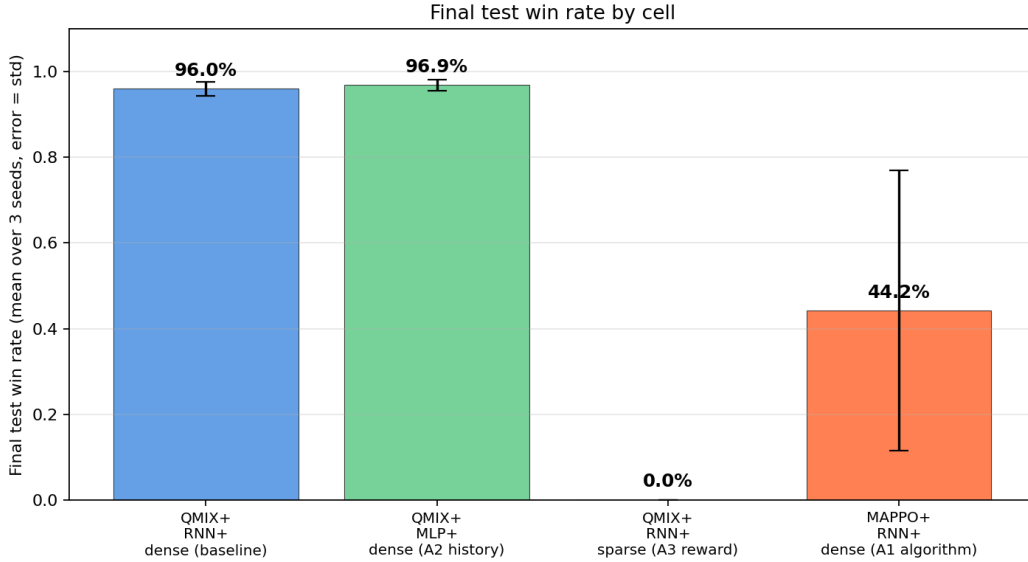


Figure 2: Final test win rate for each setting. Error bars show one standard deviation across seeds, and bar heights show the mean over three seeds. The dense-reward QMIX settings converge near ceiling performance. Sparse reward remains at zero throughout. MAPPO shows substantial variance across seeds.

settings appear in Figure 1.

Table 2: Final test win rate for each setting, reported as mean $\pm$ standard deviation across three seeds. The final column lists the individual seed results.

Cell	Mean	Std	Per-seed (1 / 2 / 3)
Baseline (QMIX+GRU+dense)	96.0%	1.6	93.8 / 97.5 / 96.9
A2 (QMIX+MLP+dense)	96.9%	1.3	95.0 / 98.1 / 97.5
A1 (MAPPO+GRU+dense)	44.2%	32.7	79.3 / 0.7 / 52.7
A3 (QMIX+GRU+sparse)	0.0%	0.0	0.0 / 0.0 / 0.0

The reward ablation produces the largest effect by a wide margin. The baseline and A3 use the same algorithm, network architecture, and training budget. The only difference is the reward definition. Final performance changes from 96.0% to 0.0%, and every sparse-reward seed fails in

the same way.

The history ablation produces no measurable difference on this scenario. A2, which replaces the GRU with an MLP, is statistically indistinguishable from the baseline and in fact has a slightly higher mean, 96.9% versus 96.0%, though the difference is well within seed-level variability. This indicates that the observation available at each timestep is already rich enough for near-optimal action selection on 3m.

The algorithm ablation shows a large difference in stability. MAPPO reaches 79.3% on its best seed, which shows that strong performance is possible. However, its second seed ends at 0.7% and its third at 52.7%, which yields a mean of 44.2% and a standard deviation of 32.7 percentage points. By comparison, QMIX reaches 96.0% with a standard deviation of 1.6. The per-seed MAPPO curves in Figure 3 show the underlying pattern: all three seeds improve substantially by about 200,000 steps, then enter a collapse phase between roughly 250,000 and 400,000 steps. One seed recovers strongly, one fails almost completely, and one finishes in an intermediate state.

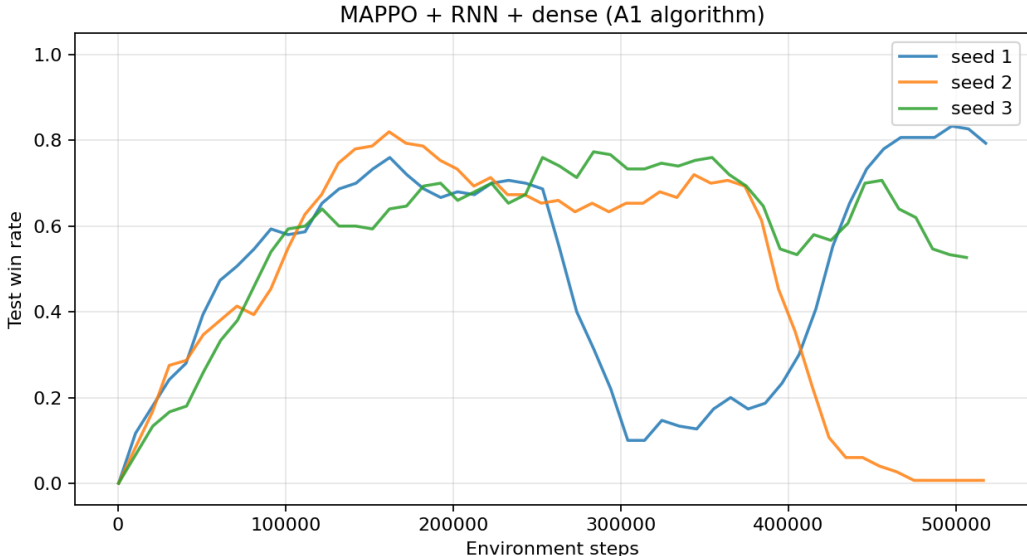


Figure 3: Per-seed MAPPO learning curves on 3m. All three seeds improve to approximately 60% to 80% win rate by about 200,000 environment steps, but each seed later experiences a substantial drop in performance. Only one seed recovers fully by the end of training.

5.3 Qualitative results

To understand why sparse reward fails so completely, we recorded evaluation statistics throughout training. Figure 4 shows the central pattern. Under sparse reward, the team’s average episode length increases from about 26 steps early in training to about 59 steps near the end. The agents therefore do learn a stable behavioral pattern. At the same time, the average number of enemies killed per episode remains close to zero throughout training. The learned behavior is therefore defensive and evasive rather than successful. The agents survive longer, but they do not attack effectively enough to win.

This outcome follows directly from the available learning signal. Under sparse reward, the only positive feedback comes from terminal victory, which random exploration almost never discovers on 3m. In contrast, defeat produces a consistent -1 signal. The resulting pressure favors policies

that postpone loss rather than policies that discover coordinated attack behavior. Dense reward changes this early training regime by assigning positive value to intermediate progress through damage dealt, which guides exploration toward combat and then toward coordinated focus fire.

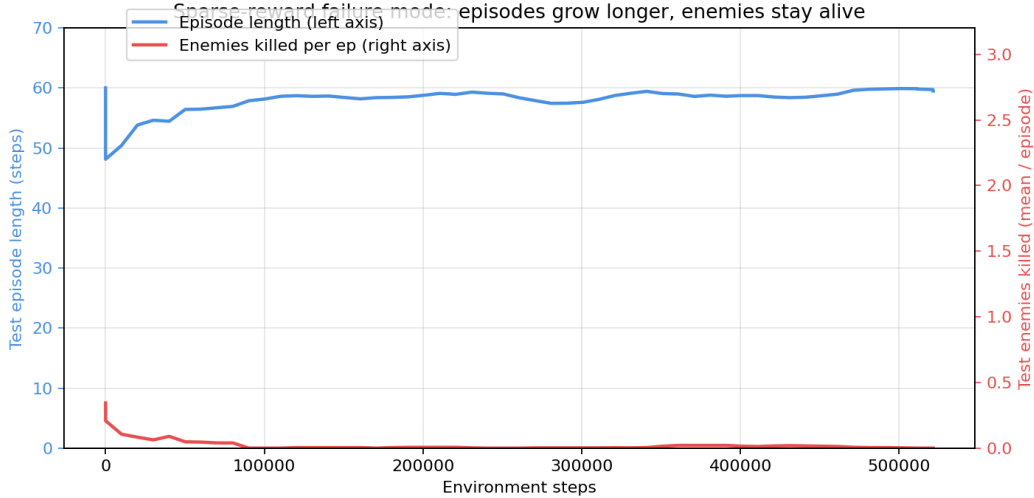


Figure 4: Failure mode under sparse reward. Mean test episode length increases from about 26 to about 59 steps over training (blue, left axis), while mean enemies killed per test episode remains near zero throughout (red, right axis). The learned policy prolongs survival but does not discover effective combat behavior.

The same pattern appears in saved replays. In the baseline setting, early checkpoints show loosely coordinated movement with occasional attacks and inconsistent outcomes, while later checkpoints show concentrated focus fire and faster victories. In the sparse-reward setting, replays remain qualitatively different throughout training. The agents drift toward the edges of the map and engage only minimally. The project artifacts include the full set of recorded replays for every setting, every seed, and three checkpoints per run.

5.4 Discussion

Three conclusions follow from these experiments.

First, reward density is the dominant determinant of learning success in this setting. On 3m with the standard SMAC defaults, the dense damage-based reward appears to provide most of the signal that makes early exploration productive. Removing that signal prevents the agents from discovering coordinated attack behavior within the available sample budget.

Second, explicit recurrence is not important on this map. Although the formal problem is partially observable, the local observation evidently contains enough information for near-optimal play in 3m. This result is specific to the scenario. Larger maps, more agents, longer-range interactions, or terrain-dependent dynamics may increase the value of temporal memory.

Third, MAPPO is much less stable than QMIX under a restricted sample budget. The best MAPPO seed demonstrates that the method can learn a strong policy, but the variance across seeds is large and the learning trajectory is fragile. QMIX, by contrast, reaches high performance consistently across all three seeds in this experiment.

5.5 Limitations

This study is intentionally narrow in scope. It covers a single SMAC scenario, a single training budget of 525,000 steps, and three seeds per experimental setting. The results therefore support conclusions about what matters most under tight compute and sample constraints on `3m`. They do not establish universal performance rankings across all cooperative MARL settings. Reward density is likely to remain important whenever successful trajectories are difficult to discover through random exploration. The value of recurrence is likely to increase on maps with stronger temporal dependence. MAPPO may also become more competitive when given substantially more training data. Those possibilities remain outside the scope of the present study.

6 Conclusion

This paper evaluated which design choices matter most for cooperative multi-agent reinforcement learning on the SMAC `3m` benchmark under a fixed budget of 525,000 environment steps. The dominant factor is the reward signal. Removing the default dense reward while holding all other components fixed reduces final test win rate from 96.0% to 0.0%. Replacing a recurrent agent network with an MLP has no measurable effect in this setting. Replacing off-policy QMIX with on-policy MAPPO leads to much larger variance across seeds and substantially lower average performance under the same budget. Taken together, these results indicate that reward design is the first issue to address when training cooperative agents on `3m`, followed by algorithmic stability. On this map, explicit recurrent memory appears to matter much less.

References

- [1] Georgios Papoudakis, Filippos Christianos, Lukas Schäfer, and Stefano V. Albrecht. Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks. In *Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*, 2021.
- [2] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2018.
- [3] Mikayel Samvelyan, Tabish Rashid, Christian Schroeder de Witt, Gregory Farquhar, Nantas Nardelli, Tim G. J. Rudner, Chia-Man Hung, Philip H. S. Torr, Jakob Foerster, and Shimon Whiteson. The starcraft multi-agent challenge. *arXiv preprint arXiv:1902.04043*, 2019.
- [4] Peter Sunehag, Guy Lever, Audrunas Gruslys, Wojciech Marian Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls, and Thore Graepel. Value-decomposition networks for cooperative multi-agent learning. In *International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS)*, 2018.
- [5] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. In *Neural Information Processing Systems (NeurIPS)*, 2022.